

review_round_record.sh

Revision: 1 **Last modified:** 2026-09-01T20:01:41Z **Authority:** vasic-digital tmux project **Maintainer:** milosvasic **Scope:** §11.4.18 script companion doc for scripts/review/review_round_record.sh and its test scripts/review/review_round_record_test.sh

Overview

A durable, append-only, machine-readable record of **one code-review round**.

It exists because a review loop that leaves no artifact on disk is a **prose-grade record of a gating decision**. §11.4.115(F) requires machine-written verdicts rather than prose; §11.4.134 requires iterate-to-clean-GO, which presumes the rounds are individually knowable; §11.4.226 holds that the *evidence class at closure* is what predicts whether the work holds.

The captured defect this closes (2026-09-01)

A code-review loop ran four rounds plus two deltas and wrote **no per-round artifact**. The round history was then stated wrongly in a commit message twice, in **opposite directions**:

Commit	Error
8dad4e3	Folded round 1 into round 2 and mis-dated a BLOCKING finding to the wrong round.
1690789	"Corrected" the above and over-rotated — moved three surviving reviewer mutations to round 1 when only one belonged there.

Both were caught only because the reviewer still held the record in its own volatile context. Both commits recorded, as an honest §11.4.6 boundary, that once that context is gone the review history is **unreconstructable from the repository**. This script removes that boundary.

Both errors were **retroactive re-attributions**, which is why append-only is the load-bearing property rather than a filing convenience.

Substrate: §11.4.116 reused, not reinvented

§11.4.116 already specifies the correct two-part channel for a long-running process an orchestrator depends on, and this script emits exactly that pair rather than inventing a parallel mechanism (§11.4.227 extend-don't-duplicate):

1. **Append-only JSONL event stream** — one event per line, never rewritten.
2. **Atomically-rewritten status snapshot** — written temp-then-rename, so a reader never observes a torn write.

The per-entry chain fields (contiguous seq + prev_digest) are the shape the constitution's own chain verifier already contracts for (constitution/scripts/gates/cm_chain_integrity_detects_alteration.sh), so the record composes with that verifier instead of competing with it.

One deliberate divergence, stated rather than silently taken:

§11.4.116's verdict vocabulary is PASS/FAIL/SKIP/OPERATOR-BLOCKED, which is the per-test-item vocabulary. A review round returns a **gate decision**, so the verdict here is the §11.4.125 / §11.4.134 vocabulary GO / NO-GO, and any other value is refused.

Prerequisites

POSIX sh, sha256sum, sed, awk, date, mktemp. No network, no build. sh -n and bash -n clean per §11.4.67.

Usage

```
scripts/review/review_round_record.sh append --stream <path> --  
round <N> \  
--reviewer <id> --verdict GO|NO-GO --evidence <path> \  

```

```

[--model <m>] [--effort <e>] \
[--finding SEVERITY:TEXT]... [--mutation ID:SURVIVED|
CAUGHT]...

scripts/review/review_round_record.sh verify --stream <path>
scripts/review/review_round_record.sh round --stream <path> --
round <N>
scripts/review/review_round_record.sh mutation --stream <path> --
mutation <ID>
scripts/review/review_round_record.sh status --stream <path>
scripts/review/review_round_record.sh --selftest

```

Worked example

```

S=qa-results/review/ATM-XXX.rounds.jsonl

# round 1 – NO-GO, one BLOCKING, one reviewer mutation that
SURVIVED
sh scripts/review/review_round_record.sh append --stream "$S" --
round 1 \
--reviewer independent-reviewer --model opus --effort high \
--verdict NO-GO --evidence qa-results/verify_round1.log \
--finding "BLOCKING:## H. over-deletion" --mutation "M-
A:SURVIVED"

# round 2 – the same mutation is now CAUGHT; round 1 stays
untouched
sh scripts/review/review_round_record.sh append --stream "$S" --
round 2 \
--reviewer independent-reviewer --model opus --effort high \
--verdict NO-GO --evidence qa-results/verify_round2.log \
--finding "IMPORTANT:regex widening" --mutation "M-A:CAUGHT"

# "which round did M-A survive in?" is now a lookup, not a memory
sh scripts/review/review_round_record.sh mutation --stream "$S" --
mutation M-A
# 1 SURVIVED
# 2 CAUGHT

```

Inputs

Flag	Required	Meaning
--stream	yes	Path to the append-only JSONL stream. Created on first append.
--round	yes (append/ round)	Positive integer. Must be strictly greater than the highest recorded round. Reviewer identity.

Flag	Required	Meaning
--reviewer	yes (append)	
--verdict	yes (append)	GO or NO-GO. Any other value is refused.
--evidence	yes (append)	Path backing the verdict. §11.4.116 forbids an evidence-less verdict event.
--model	no	Reviewer model. Defaults to the honest ? per §11.4.182 — never guessed.
--effort	no	Reviewer effort. Defaults to ? for the same reason.
--finding	no, repeatable	SEVERITY:TEXT.
--mutation	no, repeatable	ID:SURVIVED or ID:CAUGHT. Any other outcome is refused — a mutation with no observed outcome is an unfinished experiment, not a recordable fact.

Outputs

- **Stream** — one JSON object per line: seq, prev_digest, ts, round, reviewer, model, effort, verdict, evidence, findings[], mutations[], digest.
- **Snapshot** — <stream-base>.status.json, derived from the stream path so a snapshot can never point at a different stream: stream, rounds_total, last_round, last_verdict, last_seq, head_digest, updated_at.

Exit codes

Code	Meaning
0	Success.
1	Refusal, or a detected chain inconsistency.
2	Usage error.

Internal behaviour — the two seams that close the defect

Write seam. `append` refuses any round already recorded, and any round not strictly greater than the highest recorded round. A later round therefore cannot restate an earlier one. Nothing is written on a refusal path.

Read seam. `verify` recomputes every entry's digest over the exact bytes the digest covered, walks the `prev_digest` chain, and checks seq contiguity. An edit made by going *around* this tool — a text editor on the `.jsonl` — is detected.

Honest boundary (§11.4.6)

The chain proves **internal** consistency only. It cannot, by itself, detect a deletion followed by a full re-chain, nor a tail truncation: both leave a structurally perfect chain. Detecting those requires a periodic external anchor (§11.4.268), which this script **does not implement and does not claim**. What it does claim is exactly what it was built for: an attempted rewrite of an already-recorded round is refused, and an in-place edit of an earlier round is detected.

Edge cases

Case	Behaviour
Absent, unreadable, or empty stream on <code>verify</code>	REFUSE, never a clean pass. "Could not look" and "looked and found it clean" are different answers (§11.4.201(6) false null).
Predecessor entry carries no digest	Refuses to extend an unreadable chain.
<code>sha256sum</code> unavailable on <code>append</code>	Refuses to write an unchainable entry rather than writing one silently.
<code>sha256sum</code> unavailable, or silently WRONG, on <code>verify</code>	REFUSE, naming the resolved hasher path — and explicitly not reported as tampering. Both are refusals, but they demand opposite responses (repair the toolchain vs investigate an intrusion),

Case	Behaviour
	so blaming the store for a broken instrument is a §11.4.201(1) FAIL-bluff. <code>verify</code> probes the hasher against a known-answer control needle (the SHA-256 of the empty string) before walking the chain, so an instrument that resolves and emits a plausible-but-wrong digest is caught by its ANSWER, not merely by its exit status.
<code>sha256sum</code> dies part-way through a <code>verify</code> walk	The per-line guard reports REFUSE line N: digest could not be RECOMPUTED (hasher failure mid-walk ...) – NOT evidence of tampering.
Snapshot staging fails	Temp file removed; stream already durable; non-zero exit.
Findings/ mutations text containing quotes, backslashes, newlines or tabs	Escaped; entry stays one line so a pure-sh reader can parse it.

Testing

```
bash scripts/review/review_round_record_test.sh  # 23 assertions
sh      scripts/review/review_round_record.sh  --selftest
```

The test carries the two historical failure modes as fixtures:

- **T3 / T3b** — rewriting an earlier round is refused *and* the stream bytes are unchanged (failure mode (a)).
- **T5 / T5b / T5c** — a mutation recorded SURVIVED in round N stays attributable to round N after later rounds land, and round N+1 carries its own outcome (failure mode (b)).

False-positive guards (§11.4.201(1)). A recorder that refused every append would satisfy T3 perfectly and be useless. T1, T4, T6, T9 assert that a legitimate append, a later round, an untampered chain and a well-formed query all *succeed*. T14 is the same guard for the hasher probe: with no shim on PATH, the very same stream that T12/T13 refuse must still verify clean, so the probe cannot be satisfied by a `verify` that refuses unconditionally.

Self-validation (§11.4.107(10)). `--selftest` runs a golden-good chain (must verify clean) and a golden-bad tampered chain (must be detected). A verifier that only ever refused would pass the bad case and fail the good one.

Paired §1.1 mutation results

Mutation	Assertion that caught it
Both append-only guards disabled	T3 (rc=0, rewrite accepted) and T3b (stream mutated). The mutant emitted recorded round 1 (G0) seq=2 — a second, contradictory round-1 entry, i.e. the historical defect reproduced verbatim.
round query made to ignore which round was asked for	T5b — round 2's query returned round 1's record, which is exactly the (b) over-rotation.
Both verify hasher-failure discriminators stripped (the up-front known-answer probe and the per-line empty-recomputation guard)	T12 / T12b / T13 — with sha256sum shimmed to exit 127 the mutant reported digest mismatch (entry was altered after it was written) on all four entries of a healthy chain, reproducing the defect verbatim: an intact store accused of tampering because the instrument was broken.

Both mutations were restored byte-identical (sha256 verified) with zero residue markers per §11.4.84.

Related

- `scripts/review/review_round_record_test.sh` — the RED-first test.
- `constitution/scripts/gates/cm_chain_integrity_detects_alteration.sh` — the chain-shape contract this record conforms to.
- `constitution/scripts/gates/lib/execution_record.sh` — the §11.4.249 flight recorder for executed *commands*; complementary, different subject.

Not yet wired (honest status, §11.4.6)

This is a create-only delivery. The helper is **not** invoked from `scripts/verify.sh` or `scripts/pre_build_verification.sh` — wiring it in requires editing files outside this change's authorization, and the conductor does that serially.

The `.html/.pdf/.docx` twins of this document **were** produced, but not by this change: they appeared six seconds after this `.md` was written, by an export automation in the environment that this change neither invoked nor controls (UNCONFIRMED: which mechanism fired — the effect is observed, the producer is not identified here). They are faithful exports of this file's content. Recorded because an artifact this change did not author must not be silently presented as part of it (§11.4.6).

Last verified

2026-09-01T20:01:41Z — 19/19 test assertions PASS, selftest PASS, two paired mutations caught.